EXPERIMENT – 1: ExpressJS Routing, Route Parameters & Query Parameters

AIM :

To write an ExpressJS program to define routes and demonstrate handling of simple routes, route parameters, query parameters, and URL building.

SHORT DESCRIPTION :

A **route** defines how the server responds to a specific URL path.

Route parameters allow us to handle dynamic values from the URL, and **query parameters** allow filtered or optional data to be passed after the ? symbol.

PROGRAM :

```
const express = require("express");

const app = express();

// Simple Route

app.get("/", (req, res) => {

 res.send("Welcome to ExpressJS Routing");

});

// Route Parameter Example

// URL: /user/101

app.get("/user/:id", (req, res) => {

 const userId = req.params.id;

 res.send("User ID is: " + userId);

});
```

```
// Query Parameter Example

// URL: /search?item=laptop

app.get("/search", (req, res) => {

 const itemName = req.query.item;

 res.send("You searched for: " + itemName);

});

// URL Building (Redirecting)

app.get("/go-to-user", (req, res) => {

 const id = 500;

 res.redirect(`/user/${id}`);

});

// Start Server

app.listen(3000, () => {

 console.log("Server started on port 3000");

});
```

EXPERIMENT – 2: HTTP Methods – Accept, Retrieve & Delete Data

AIM :

To write an ExpressJS program to accept data using POST, retrieve data using GET, and delete a specific resource using DELETE HTTP methods.

DESCRIPTION :

HTTP methods define how clients interact with a server.
In this experiment, we use:

- **POST** → to accept and store data
- **GET** → to retrieve stored data
- **DELETE** → to delete a resource based on an ID

```
const express = require("express");
const app = express();

app.use(express.json());   // To read JSON data sent from client

// Temporary in-memory array (acts like a database)
let students = [];



// POST - Accept Data
// URL: /students  (Body: { "id":1, "name":"Suresh" })
app.post("/students", (req, res) => {
 const data = req.body;
 students.push(data);
 res.send("Student data added successfully");
});
```

```
// GET - Retrieve All Data
// URL: /students
app.get("/students", (req, res) => {
 res.send(students);
});
// DELETE - Delete Specific Resource
// URL: /students/1
app.delete("/students/:id", (req, res) => {
 const id = req.params.id;

 students = students.filter(student => student.id != id);

 res.send("Student with ID " + id + " deleted");
});
// Start Server
app.listen(3000, () => {
 console.log("Server started on port 3000");
});
```

EXPERIMENT – 3: Program Using Templating Engine (EJS)

AIM:

To write an ExpressJS program using a templating engine (EJS) to display dynamic data inside an HTML page.

DESCRIPTION :

A templating engine allows us to embed JavaScript values directly inside HTML pages.
ExpressJS supports many templating engines, and in this experiment, we use EJS (Embedded JavaScript Templates).
EJS helps render webpages dynamically by passing data from the server to the HTML template.

PROGRAM :

app.ejs :

```
const express = require("express");
const app = express();

// Set EJS as the templating engine
app.set("view engine", "ejs");

// Route to render template
app.get("/", (req, res) => {
 const message = "Welcome to EJS Templating Engine!";
 res.render("home", { msg: message });
});

// Start Server
app.listen(3000, () => {
 console.log("Server started on port 3000");
});
```

home.ejs :

```
<!DOCTYPE html>
<html>
<head>
   <title>EJS Example</title>
</head>
<body>
   <h1><%= msg %></h1>
   <p>This message is displayed using EJS templating engine.</p>
</body>
</html>
```

EXPERIMENT – 4: Program to Work with Form Data

AIM:

To write an ExpressJS program to accept form data submitted by the user and display the submitted information using a templating engine (EJS).

DESCRIPTION :

Form data refers to the input values entered by a user inside an HTML form.
To handle this form data, ExpressJS uses the middleware **express.urlencoded()**, which reads the values entered in the form.

PROGRAM :

```
const express = require("express");
const app = express();

// Middleware to read form data
app.use(express.urlencoded({ extended: true }));

// Route to display HTML form
app.get("/", (req, res) => {
 res.send(`
   <h2>User Registration Form</h2>
   <form method="POST" action="/submit">
     <label>Name:</label>
     <input type="text" name="name" required /><br><br>

     <label>Email:</label>
     <input type="email" name="email" required /><br><br>

     <button type="submit">Submit</button>
   </form>
 `);
});
```

```
// Route to handle submitted form data
app.post("/submit", (req, res) => {
 const name = req.body.name;
 const email = req.body.email;

 res.send(`
   <h2>Form Submitted Successfully</h2>
   <p><strong>Name:</strong> ${name}</p>
   <p><strong>Email:</strong> ${email}</p>
 `);
});

// Start server
app.listen(3000, () => {
 console.log("Server running on port 3000");
});
```

EXPERIMENT – 5: Session Management using Cookies and Sessions

AIM :

To write an ExpressJS program to demonstrate session management using cookies and sessions, where a session is created, accessed, and destroyed.

DESCRIPTION :

Session management ensures that user data is stored temporarily during their visit to a web application.
A **cookie** stores a unique session ID in the browser, and a **session** stores user information on the server.
Using the `express-session` and `cookie-parser` middleware, we can create a session, store data in it, retrieve session data, and destroy the session.

PROGRAM :

```
const express = require("express");
const session = require("express-session");
const cookieParser = require("cookie-parser");

const app = express();

// Middleware for cookies
app.use(cookieParser());

// Session configuration
app.use(
 session({
   secret: "mySecretKey",        // used to sign the session ID cookie
   resave: false,
   saveUninitialized: true,
   cookie: { maxAge: 60000 }     // cookie expires in 1 minute
 })
```

```
);

// Route 1: Create Session
app.get("/login", (req, res) => {
 req.session.username = "Suresh"; // store data in session
 res.send("Session Created! Cookie Stored in Browser.");
});

// Route 2: Access Session Data
app.get("/profile", (req, res) => {
 if (req.session.username) {
   res.send("Welcome " + req.session.username + " — Session Active!");
 } else {
   res.send("No active session found. Please login first.");
 }
});

// Route 3: Destroy Session
app.get("/logout", (req, res) => {
 req.session.destroy(() => {
   res.send("Session destroyed. Logged out successfully.");
 });
});

// Start Server
app.listen(3000, () => {
 console.log("Server running on port 3000");
});
```

EXPERIMENT – 6: User Authentication using ExpressJS & Sessions

AIM :

To write an ExpressJS program to authenticate a user using username and password, and maintain login state using sessions.

DESCRIPTION:

We create a login page where the user enters a username and password.
The server checks the credentials, and if they are valid, a session is created using **express-session**.
Authenticated users can access the dashboard; others cannot.

PROGRAM :

```
const express = require("express");
const session = require("express-session");
const app = express();

// Middleware to read form data
app.use(express.urlencoded({ extended: true }));

// Session setup
app.use(
 session({
   secret: "secret",
   resave: false,
   saveUninitialized: true
 })
);

// Login Page
app.get("/login", (req, res) => {
```

```
 res.send(`
   <h2>Login</h2>
   <form method="POST" action="/login">
     <input name="username" placeholder="Username" required />
     <br><br>
     <input name="password" type="password" placeholder="Password"
required />
     <br><br>
     <button type="submit">Login</button>
   </form>
 `);
});

// Login Check
app.post("/login", (req, res) => {
 const { username, password } = req.body;

 if (username === "admin" && password === "1234") {
   req.session.user = username;
   return res.send("Login Successful! <br><a href='/dashboard'>Go to
Dashboard</a>");
 }

 res.send("Invalid Login! <a href='/login'>Try Again</a>");
});

// Protected Dashboard
app.get("/dashboard", (req, res) => {
 if (req.session.user) {
   return res.send(`Welcome ${req.session.user}! <br><a
href='/logout'>Logout</a>`);
 }
 res.send("Access Denied! Please login first.");
});
```

```
// Logout
app.get("/logout", (req, res) => {
 req.session.destroy(() => {
   res.send("Logged Out! <a href='/login'>Login Again</a>");
 });
});

app.listen(3000, () => console.log("Server running on port 3000"));
```

EXPERIMENT – 7: MongoDB Connection and CRUD Operations using Mongoose

AIM :

To write an ExpressJS program to connect to a MongoDB database using Mongoose and perform CRUD operations (Create, Read, Update, Delete) on a collection.

DESCRIPTION :

Mongoose is an Object Data Modeling (ODM) library used in Node.js to simplify work with MongoDB.
It allows applications to:

- Connect to MongoDB easily
- Create schemas and models
- Perform CRUD operations through simple functions

PROGRAM :

```
const express = require("express");
const mongoose = require("mongoose");
const app = express();
app.use(express.json());
// 1. Connect to MongoDB
mongoose
 .connect("mongodb://127.0.0.1:27017/studentdb")
 .then(() => console.log("MongoDB Connected"))
 .catch(err => console.log(err));
// 2. Define Schema
const studentSchema = new mongoose.Schema({
 name: String,
 age: Number,
 branch: String
});
```

```
// 3. Create Model (Collection = students)
const Student = mongoose.model("Student", studentSchema);
// CREATE (POST) - Insert a student
app.post("/students", async (req, res) => {
 const data = req.body;
 const student = await Student.create(data);
 res.send(student);
});
// READ ALL (GET) - Retrieve all students
app.get("/students", async (req, res) => {
 const students = await Student.find();
 res.send(students);
});
// READ ONE (GET) - Retrieve student by ID
app.get("/students/:id", async (req, res) => {
 const student = await Student.findById(req.params.id);
 res.send(student);
});
// UPDATE (PUT) - Update student by ID
app.put("/students/:id", async (req, res) => {
 const updatedStudent = await Student.findByIdAndUpdate(
   req.params.id,
   req.body,
   { new: true }        // return updated data
 );
 res.send(updatedStudent);
});
// DELETE (DELETE) - Remove student by ID
app.delete("/students/:id", async (req, res) => {
 await Student.findByIdAndDelete(req.params.id);
 res.send("Student deleted successfully");
});

// Start server
app.listen(3000, () => console.log("Server running on port 3000"));
```